

Table Constraints in Clause Learning CSP Solvers

Ozan Erdem¹, George Katsirelos², and Fahiem Bacchus¹

¹ Department of Computer Science, University of Toronto,
Toronto, Ontario, Canada, M5S 3H5,
{ozan, fbacchus}@cs.toronto.edu

² MIAT, INRA, Toulouse, France,
george.katsirelos@toulouse.inra.fr

Abstract. We investigate alternative methods for implementing table constraints in clause learning CSP solvers (CL solvers). CL solvers have been an important development in CP solving as they can provide important performance improvements on some problems. Furthermore, table constraints remain an important and useful modeling tool in CP. Hence, it is important to be able to utilize table constraints in CL solvers effectively. Here we compare different ways of achieving GAC propagation over table constraints in a CL solver. These methods require different representations of the constraint. First we utilize a CNF encoding of the table constraint which has the property that unit propagation achieves GAC. We compare this with the use of a traditional GAC propagation algorithm for tables, Simple Tabular Reduction (STR). To utilize STR in a CL solver we also develop a method for extracting clausal explanations for pruned values. We also develop and test a negative version of STR which more compactly represents tables that have fewer falsifying than satisfying tuples. This version also generates clausal explanations. We implement these different methods in the CL solver minicsp, and compare them to the CNF encodings. Even though our proposed methods do not perform as well as the encodings in general, we believe that they will stimulate further research on this area.

1 Introduction

A number of robust and powerful CSP solvers have been developed and are publicly available. A recent development in CSP solver technology has been the importation of ideas from SAT, specifically clause learning [9, 10, 12, 13]. Clause learning improves the theoretical power of a CSP solver, and recently well engineered clause learning CSP solvers (CL solvers) have demonstrated very good empirical performance on a range of problems (see, e.g., the results of recent MiniZinc Challenges, <http://www.minizinc.org/>).

One of the main technologies exploited by CL solvers is the ability to generate clausal explanations from global constraints. Specifically, when a global constraint prunes a domain value, a clause justifying that pruning needs to be

generated (also called an explanation). Once the solver can obtain a clause for each pruned value, it can perform clause learning much like a regular SAT solver. In particular, when a contradiction is detected the CL solver must be able to obtain the clausal reasons for the various domain prunings that lead up to the contradiction, and resolve these reasons against the base contradiction.

A considerable amount of work has been done showing how to generate clausal reasons (typically on-demand or lazily) from propagators for various global constraints, e.g., [9, 14, 5, 4]. Methods for generating explanations from table constraints were examined in [9], but from a practical point of view these methods have not been previously examined in the light of the performance tradeoffs of modern CL solvers.

Table constraints are of course very important in constraint programming. They are easy for inexperienced users to use, problem domains often contain special ad-hoc constraints that are most easily encoded extensionally as table constraints, and information stored in databases is often accessed most conveniently as a table constraint. It is not surprising then that much research has been done on efficiently achieving GAC on table constraints.

In this paper we examine some different options for implementing table constraints in a CL solver so as to efficiently achieve GAC. We look at a clausal decomposition, i.e., representing the table constraint as a set of clauses (CNF). The main inference method available on clauses in a CL solver is unit propagation, hence we examine a clausal encoding on which unit propagation is sufficient to achieve GAC. A CL solver can also treat a table constraint as a black box—just as if it was any other propagator for achieving GAC over a specific type of constraint. As mentioned above, all that the CL solver needs is the ability to generate (or obtain on the fly) clausal reasons for the values pruned by the constraint. We examine one of the most efficient and simplest “propagators” for table constraints, Simple Tabular Reduction [11]. Although not typically viewed as being a propagator, an STR represented table constraint interfaces with a CL solver in the same way that any other propagator would. Finally, we examine a new STR-like algorithm for propagating table constraints represented by their falsifying rather than satisfying tuples. Negative STR can be more effective for table constraints that have many more satisfying tuples than falsifying tuples. We give an algorithm for achieving GAC on this negative STR table. We compare these different ways of handling table constraints in CL solvers and draw some conclusions about their relative effectiveness in practice.

2 Background

A constraint problem $\mathcal{P}$ consists of a finite set $\mathcal{V} = \{V_1, \dots, V_n\}$ of variables, each with a finite domain of values $Dom[V_i]$ and a finite set of constraints $\mathcal{C} = \{c_1, \dots, c_m\}$. Each constraint c_i is defined over some subset of variables, $scope(c_i) \subseteq \mathcal{V}$. An assignment $\mathcal{A}$ is a set of variable value assignments $\{V^1 = d^1, \dots, V^k = d^k\}$ in which any variable is assigned at most a single value. Let

$vars(\mathcal{A}) \subseteq \mathcal{V}$ denote the set of variables assigned in $\mathcal{A}$. $\mathcal{A}$ is said to **cover** a constraint c if $scope(c) \subseteq vars(\mathcal{A})$.

A constraint c can be viewed as a Boolean function from assignments $\mathcal{A}$ that cover it to *true* (in which case $\mathcal{A}$ is said to be satisfying) or *false* ($\mathcal{A}$ is said to be falsifying). c is actually a function only of the variables in $scope(c)$, so if $\mathcal{A}$ covers c and $V \notin scope(c)$, then $\mathcal{A}$ with any assignments to V removed will be mapped to the same value as $\mathcal{A}$.

Often we consider $scope(c)$ to be ordered, and then for assignments $\mathcal{A}$ such that $vars(\mathcal{A}) = scope(c)$ we can order its variable assignments in the same way. Then without loss of information we can remove the variables leaving only a sequence of values. This ordered sequence of values is called a **tuple** for c . If $\mathcal{A}$ is satisfying then its corresponding tuple is said to be a positive tuple (p-tuple) for c , otherwise it is a negative tuple for c (n-tuple). If t is a tuple we let $vars(t)$ denote the variables t specifies values for. If $V \in vars(t)$ we let $t[V]$ denote the value assigned to V in t , and we say that t is **valid** iff for all $V \in vars(t)$ we have that $t[V] \in Dom[V]$. A valid p-tuple t for c is said to be a **support** for the value $d \in Dom[V]$ (where $V \in scope(c)$) if $t[V] = d$.

A positive table constraint (p-table) is a constraint that is specified by a set of p-tuples, a negative table constraint is specified by a set of n-tuples. In particular these sets of tuples are complete: t satisfies a p-table constraint T if and only if $t \in T$ (similarly for n-tables).

Finally, a constraint c is said to be generalized arc-consistent (GAC) if $\forall V \in scope(c), d \in Dom[V]$ there exists a valid p-tuple t for c such that $t[V] = d$.

A *propositional variable* is a variable with domain $\{0, 1\}$. By convention, if x is a propositional variable, we write x for $x = 1$ and $\neg x$ for $x = 0$. x and $\neg x$ are *literals* of the variable x and they are called complementary to each other. If l is a literal, we denote its complement by $\bar{l}$. A *clause* is a constraint which is a set of literals, interpreted as their logical disjunction, e.g., $(x, y, \bar{z})$, which is interpreted as $x \vee y \vee \neg z$.

In a CL solver, for each variable we maintain (implicitly or explicitly) a *clausal encoding* of its domain. There are several options for this; we use the order encoding in [12, 3, 1]. For every multi-valued variable V with $Dom[V] = \{d_1, \dots, d_k\}$ we have k Boolean assignment variables $A_{V=d_j}$ and $k+1$ *order* variables³ $A_{V \leq d_j}$, $0 \leq j \leq k$. The variable $A_{V=d_j}$ is true when $V = d_j$, and is false when d_j has been pruned from V 's domain. The variable $A_{V \leq d_j}$ is true when all values greater than d_j have been pruned from the domain of V and false when all values less than or equal to d_j have been pruned. We also have $O(k)$ clauses that encode $A_{V=d} \iff A_{V \leq d} \wedge \neg A_{V \leq d-1}$ and $A_{V \leq d} \rightarrow A_{V \leq d+1}$ and the unit clauses $(A_{V \leq d_k})$ and $(\neg A_{V \leq d_0})$. These ensure that each variable is assigned exactly one value.

³ d_0 is a sentinel value, which we use here only to simplify the exposition. In practice it is optimized away.

3 Clausal Decompositions of Table Constraints

We aim to examine methods for implementing table constraints in CL solvers. Such solvers have efficient mechanisms for handling clauses, as they can learn many clauses during solving, and it is very easy to use the same mechanisms to deal with an initial set of input clauses. Hence, one way of implementing table constraints is to convert them into a set of input clauses. The only restriction is that the solver only has access to unit propagation (UP) to reason about these clauses. Thus to achieve GAC we must use an encoding on which UP achieves GAC. One significant advantage of clausal encodings is that the clausal reasons are already available—no extra computation is needed to obtain them.

3.1 Support Tuple Encoding

In [2] a CNF encoding for a table constraint c was given on which UP achieves GAC. This encoding was an adaptation of encodings presented in [8].

To encode the constraint c we utilize additional propositional variables $t_1, \dots, t_m$, each one representing one of the m different p-tuples of c . Let τ_i be the p-tuple represented by the propositional variable t_i . Using the t_i variables we can write the clauses capturing C as follows. For the variable $V \in \text{scope}(c)$ and value $d \in \text{Dom}[V]$, let $\{s_1, \dots, s_i\}$ be the subset of $\{t_1, \dots, t_m\}$ such that the satisfying tuples represented by the s_i are precisely the set of tuples τ_i such that $\tau_i[V] = d$ (these are the supports of $V = d$). For each variable and value $V = d$ we have the clause $(s_1, \dots, s_i, \neg A_{V=d})$ ($V = d$ must be false if it has no support). Finally, we have for each p-tuple of C , τ_i , and assignment $V = d \in \tau_i$ the clause $(A_{V=d}, \neg t_i)$, which captures the condition that the tuple of assignments τ_i cannot hold if $V = d$ cannot be true and also the condition that if τ_i holds then so do all of its variable assignments.

For example consider the table constraint C , where $\text{scope}(C) = \{V_1, V_2\}$ and $\text{Dom}[V_1] = \text{Dom}[V_2] = \{1, 2\}$. Let the table be represented by a set of tuples as $\{(1, 1), (2, 1), (2, 2)\}$. To encode this constraint we associate a new variable with each row, namely t_1, t_2 and t_3 . We have the following clauses: $(t_1, \neg A_{V_1=1})$, $(t_2, \neg A_{V_1=2})$, $(t_1, t_2, \neg A_{V_2=1})$, $(t_3, \neg A_{V_2=2})$ for capturing the loss of supports for each variable value pair. In addition, we have the following clauses to capture the relation between assignments and the new variables: $(A_{V_1=1}, \neg t_1)$, $(A_{V_2=1}, \neg t_1)$, $(A_{V_1=2}, \neg t_2)$, $(A_{V_2=1}, \neg t_2)$, $(A_{V_1=2}, \neg t_3)$, $(A_{V_2=2}, \neg t_3)$.

It has been shown [2] that this encoding enforces GAC on a table constraint. It can also be observed that the size of this encoding (i.e., the sum of the lengths of the clauses) is linear in the size of the constraint's p-table representation: $O(mr)$ where m is the number of satisfying tuples, and r is the constraint's arity. Hence UP on this encoding will operate in time linear in the size of the constraint's p-table representation.

4 Clausal Reasons from Table Propagators

In this section we examine an algorithmic representation for achieving GAC on a table constraint. There are many such algorithms, but in this paper we focus on simple tabular reduction, a GAC algorithm that is particularly efficient under backtracking. When we implement table constraints using non-clausal representations the CL solvers must be able to derive clausal reasons for pruned values. First we discuss the standard STR algorithm that works on p-table constraints. Then we turn our attention to a negative version of STR that works on n-table constraints. Our negative STR algorithm is new and might be of interest beyond the context of CL solvers.

4.1 Positive STR

Simple Tabular Reduction (STR) is an efficient GAC algorithm which dynamically maintains tables in order to keep track of supports. It was first introduced by [15], and used in the context of a backtracking search algorithm by [11] along with a number of optimizations under the name STR2+. In this paper we will refer to STR2+ as *positive STR*, and as *STR* when the context is clear. In positive STR, the table t of a constraint is divided into its upper and lower parts called *top*(t) and *bottom*(t) with all the tuples in *bottom* being invalid.

As described in [11] achieving GAC with STR involves processing the tuples in *top* to determine if they are valid (these tuples become invalid as the values they assign to variables are pruned). If a tuple t is found to be invalid it is moved to *bottom*, while if t is found to be valid all of the values it assigns are marked as having a support (i.e., these values are GAC). After all tuples in *top* are processed, the unsupported variable values are pruned. The key contribution of STR is that the invalid tuples need never be examined, the valid tuples need be examined only once, and backtracking can be achieved by simply restoring the variable domains and moving the marker that divides *top* from *bottom*.

Our addition to STR is to compute clausal reasons for the values pruned. These reasons can also be computed on demand (lazily). This is achieved as follows. When processing the tuples in *top*, for any invalid tuple t we detect, we remember the variable value pair that made t invalid. This is some $V = d$ such that $t[v] = d$ and $d \notin \text{Dom}[V]$. There must be at least one such pair in t for it to be classified as invalid. If there are multiple pairs we simply choose the first one found. Hence, each tuple in *bottom* is marked with a proposition $A_{V_i=d_i}$. Then if we prune a value, e.g., $V = d$ to compute a reason we scan all tuples in *bottom*, locate those tuples t with $t[V] = d$ and accumulate their propositional reasons into a set. The conjunction of these propositions then implies the loss of all supporting tuples for $V = d$, and thus implies $\neg A_{V=d}$. This implication is a clause and supplies the reason we wanted to compute. In [7] an alternative method is presented for computing reasons from table constraints, where they use a trie containing tuples of the table. This trie is built using a static ordering of the variables and hence some extra processing is required to extract a reason

Algorithm 1: Initialize n-STR

```

1 init-nSTR (c);
2 dprod =  $\prod_{V \in \text{scope}(c)} |\text{Dom}[V]|$ ;
3 foreach  $X \in \text{scope}(c)$  do
4   c.dprod[X] = dprod /  $|\text{Dom}[X]|$ ;
5   foreach  $d \in \text{Dom}[X]$  do
6     // Initialize Forbidden Tuple Counts
7     c.ftc[X][d]  $\leftarrow$  # of n-tuples  $t$  for  $c$  with  $t[X] = d$ ;
8     if c.ftc[X][d] == c.dprod[X] then
9       prune (X,d);
10      c.update = true;
11 if c.update then
12   prop-nSTR (c);

```

compatible with the order of the variables along the current search path. Our approach more naturally yields a reason compatible with the current search path.

Since we must scan all tuples in *bottom* computing reasons can be fairly expensive, and it can be done lazily by simply restoring the marker between *top* and *bottom* to its state when the value was pruned and then examine all tuples in *bottom*. Alternatively, by using more memory these reasons could be accumulated as the tuples are detected to be invalid. We did not experiment with this alternative, as the STR approach in general is not that promising for CL solvers (see Sec. 5).

4.2 Negative STR

The new STR-based algorithm that we describe for negative table constraints c reuses the table data structures of the positive STR. In addition, it maintains the following data structures:

- **dprod**: is an array of size $r = |\text{scope}(c)|$, such that **dprod**[i] is the number of valid (satisfying or falsifying) tuples that contain each value of variable i .
- **ftc**: is a two dimensional array giving the number of valid falsifying tuples for every variable/value pair.

The initialization of the n-STR propagator is shown in Algorithm 1 and the propagator itself in Algorithm 2. They both compute **dprod** and **ftc** in a straightforward manner. The only complication is that when we prune a value, the counts have to be recomputed from scratch, hence we set the variable *update* to *true* in order to repeat the loop.

To see the correctness of n-STR, observe that it computes exactly for each $X = d$, the size of the set $\mathcal{V}(X = d)$ of valid tuples t with $t[X] = d$ and the size of the set $\mathcal{F}(X = d)$ of valid falsifying tuples t with $t[X] = d$. We see that $\mathcal{F}(X = d) \subseteq \mathcal{V}(X = d)$ and moreover the set of supporting tuples $\mathcal{S}(X = d)$ satisfies $\mathcal{S}(X = d) \subseteq \mathcal{V}(X = d)$, $\mathcal{F}(X = d) \cup \mathcal{S}(X = d) = \mathcal{V}(X = d)$ and $\mathcal{S}(X = d) \cap \mathcal{F}(X = d) = \emptyset$. Therefore $|\mathcal{S}(X = d)| = |\mathcal{V}(X = d)| - |\mathcal{F}(X = d)|$.

Algorithm 2: Achieve GAC on an n-STR

```

1 prop-nSTR (c);
  // Propagate The Pruned Values
2 while c.update do
3   c.update = false;
4   dprod =  $\prod_{V \in \text{scope}(c)} |\text{Dom}[V]|$ ;
5   foreach  $X \in \text{scope}(c)$  do
6     | c.dprod[X] = dprod /  $|\text{Dom}[X]|$ ;
7   foreach n-tuple nT in c.top do
8     | if  $\neg \text{valid}(nT)$  then
9       |   foreach  $X \in \text{scope}(c)$  do
10        |   | c.ftc[X][nT[X]] -= 1;
11        |   // Move nT to inactive part of c
12   foreach  $X \in \text{scope}(c), d \in \text{Dom}[X]$  do
13     | if c.ftc[X][d] == c.dprod[X] then
14       |   prune (X, nT[X]);
15       |   c.update = true;

```

We must prune $X = d$ when $|\mathcal{S}(X = d)|$ reaches 0, or $|\mathcal{V}(X = d)| = |\mathcal{F}(X = d)|$. But $|\mathcal{V}(X = d)| = \mathbf{dprod}[X]$ and $|\mathcal{F}(X = d)| = \mathbf{ftc}[X][d]$, so since these counts are maintained correctly, n-STR is sound and enforces GAC.

Let r be the size of the constraint's scope, d the size of the maximum variable domain and T the number of n-tuples in the constraint. The complexity of computing **dprod** is $O(r)$ per iteration (lines 4–6) and the complexity of updating **ftc** is $O(rT)$. The loop can be executed at most $O(rd)$ times, but this is cumulative over an entire branch. This gives a total cumulative complexity over a branch of $O(r^2 + r^2dT)$. Crucially, this complexity is linear in T , although it is worse than that of STR and other algorithms for positive table constraints.

To compute clausal reasons from n-STR is more complex to implement. We chose to implement a simple but non-minimal way of computing reasons. If n-STR c prunes $V = d$ then it is easy to see that the conjunction of all of the other pruned values for the other variables in c 's scope is a sufficient reason. For example, if $\text{scope}(c) = \{V_1, V_2, V_3\}$, and the value $V_2 = a$, $V_2 = d$, and $V_3 = a$ have been pruned from V_2 and V_3 at the time c prunes $V_1 = d$ (note these values could be, and probably are, pruned by other constraints), then we know that $\neg V_2 = a \wedge \neg V_2 = d \wedge \neg V_3 = a \rightarrow \neg V_1 = d$. This can be encoded in a clause using the $A_{V=d}$ variables. These reasons are not as compact as those produced by positive STR.

5 Empirical Results

To evaluate these three methods, we have implemented them in the CL solver `minicsp` (<http://www7.inra.fr/mia/T/katsirelos/minicsp.html>). The evaluation was performed using XCSP benchmarks (<http://www.cril.univ-artois.fr/~lecoutre/benchmarks.html>). All experiments were performed on a Intel Core 2 Duo, 2.00 GHz.

We experimented with a set of 163 real instances of consisting of mostly crossword puzzles (`lexPuzzle`, `ukPuzzle`, `ogdPuzzle`, `ogdHerald`, `lexHerald`, `driver`, `renault`), a set of 757 patterned instances consisting of mostly graph isomorphism problems (`si2-m4D`, `si4-m4D`, `si4-bvg`, `si6-m4D`, `si6-bvg`, `BH-4-4`, `BH-4-7`) and a set of 50 randomly generated instances to evaluate the different methods for implementing p-table constraints in `minicsp`.

Figure 1 reports the number of solved real instances and Figure 2 reports the number of solved patterned instances using the CNF support tuple encoding (STE) and the positive STR method (p-STR).⁴ We see that although on the real instances the performance is similar, on the patterned instances the clausal STE encoding performs much better (solving more problems in less time).

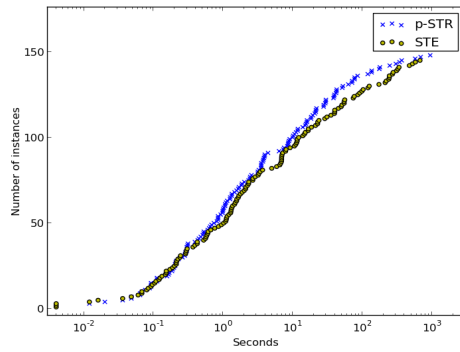


Fig. 1. Real instances

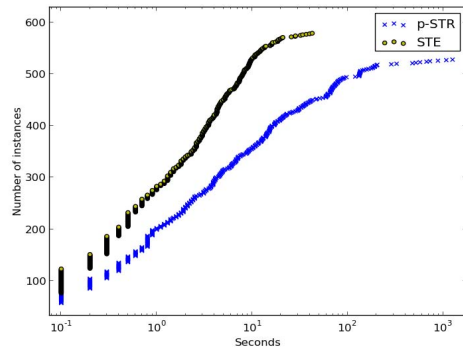


Fig. 2. Patterned instances

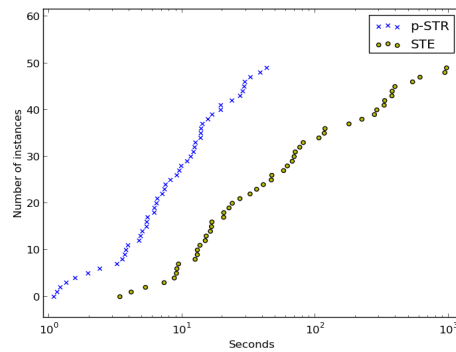


Fig. 3. Random instances

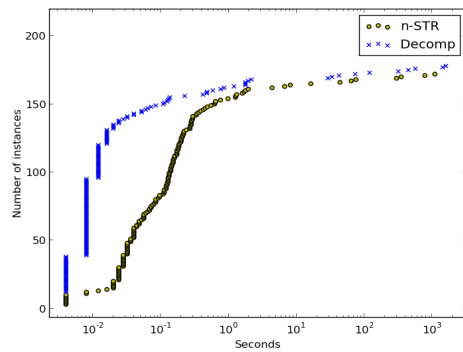


Fig. 4. Instances with negative tables

⁴ In addition to the support tuple encoding we also experimented with a MDD clausal encoding. However, we do not report results for this since it was not competitive with the other methods.

The 50 random instances all consisted of 20–25 positive table constraints, each with arity between 5–10 variables, each with domain size 8–9. Each tuple of values was inserted as a p-tuple into the table with probability 0.5, so that these constraints had a tightness of approximately $1/2$. Figure 3 shows the comparison between our p-STR implementation and the support tuple encoding (STE).

The results reveal that the support tuple encoding shows the best performance for real and patterned instances. A special purpose propagator such as STR yields better results when the table constraints tend to be tight as seen on the random problems. However, none of the XCSP benchmarks we used contain tables with a tightness close to 50%.

Although it is known that the crossword instances lack an underlying structure which can be exploited by conflict analysis [6] they remain an apt choice as benchmark, since a clause learning solver must generate reasons from all of the problems’ constraints. Profiling of STR shows that in the crossword instances 60% of the runtime is spent extracting clausal reasons while pruning values. Time spent on this extraction can be as high as 70% in the patterned instances. However, this does not account for the performance difference in patterned instances, as seen in the logarithmic gap between STR and STE is shown in Figure 2. This gap demonstrates that STE would still be faster even if the reasons from STR could be generated for free. We could further confirm this by verifying that the solver generates fewer conflicts when using STE, on average. We conjecture that the additional propositions in the STE encoding allows the solver to generate smaller proofs.

Finally, to evaluate our n-STR approach we experimented on 182 patterned instances (`bqwh-15-106`, `travellingSalesman-20`, `QCP-10`, `cril`, `QWH-10`, `coloring`) specified with n-tables. Since the support tuple encoding works with p-tuples, we tested the n-STR algorithm against a naive encoding where each conflicting tuple is directly represented as a clause. Figure 4 shows the number of solved instances using these two methods, where *n-STR* stands for negative STR and *Decomp* stands for the naive encoding. For n-STR we see that it is generally inferior to the naive encoding on this problem set.

In general, it would seem that once the simplicity of using the clausal encoding is considered for CL solvers a CNF encoding will probably be the best choice for representing table constraints. The results could change, however, depending on future developments: (1) potentially a more memory expensive way of computing better reasons from p-STR tables might benefit p-STRs; (2) better reasons could potentially be computed from n-STRs. These are useful questions for future research.

References

1. Ansótegui, C., Manyà, F.: Mapping problems with finite-domain variables to problems with boolean variables. *Theory and Applications of Satisfiability Testing* pp. 899–899 (2005)
2. Bacchus, F.: Gac via unit propagation. In: *Principles and Practice of Constraint Programming (CP)*. pp. 133–147. Springer-Verlag, New York (2007)

3. Crawford, J.M., Baker, A.B.: Experimental results on the application of satisfiability algorithms to scheduling problems. In: In Proceedings of the Twelfth National Conference on Artificial Intelligence. pp. 1092–1097 (1994)
4. Downing, N., Feydy, T., Stuckey, P.J.: Explaining flow-based propagation. In: Integration of AI and OR Techniques in Constraint Programming for Combinatorial Optimization Problems (CPAIOR). pp. 146–162 (2012)
5. Gange, G., Stuckey, P.J.: Explaining propagators for s-dnnf circuits. In: Integration of AI and OR Techniques in Constraint Programming for Combinatorial Optimization Problems (CPAIOR). pp. 195–210 (2012)
6. Gange, G., Stuckey, P.J., Szymanek, R.: Mdd propagators with explanation. *Constraints* 16(4), 407–429 (Oct 2011), <http://dx.doi.org/10.1007/s10601-011-9111-x>
7. Gent, I.P., Miguel, I., Moore, N.C.A.: Lazy explanations for constraint propagators. In: Proceedings of the 12th international conference on Practical Aspects of Declarative Languages. pp. 217–233. PADL’10, Springer-Verlag, Berlin, Heidelberg (2010), http://dx.doi.org/10.1007/978-3-642-11503-5_19
8. Hebrard, E., Bessière, C., Walsh, T.: Local consistencies in sat. In: Proceedings of Theory and Applications of Satisfiability Testing (SAT). pp. 400–407 (2003)
9. Katsirelos, G.: Nogood Processing in CSPs. Ph.D. thesis, University of Toronto (2008)
10. Katsirelos, G., Bacchus, F.: Generalized nogoods in csps. In: Proceedings of the AAAI National Conference (AAAI). pp. 390–396 (2005)
11. Lecoutre, C.: Str2: optimized simple tabular reduction for table constraints. *Constraints* 16(4), 341–371 (2011)
12. Ohrimenko, O., Stuckey, P.J., Codish, M.: Propagation = lazy clause generation. In: Principles and Practice of Constraint Programming (CP). pp. 544–558 (2007)
13. Ohrimenko, O., Stuckey, P.J., Codish, M.: Propagation via lazy clause generation. *Constraints* 14(3), 357–391 (2009)
14. Schutt, A., Feydy, T., Stuckey, P.J., Wallace, M.G.: Explaining the cumulative propagator. *Constraints* 16(3), 250–282 (2011)
15. Ullmann, J.R.: Partition search for non-binary constraint satisfaction. *Inf. Sci.* 177(18), 3639–3678 (Sep 2007), <http://dx.doi.org/10.1016/j.ins.2007.03.030>